The Digital Ear - Part 1: Quick Start Guide

A detailed implementation guide is located here: The Digital Ear
GUI Demo here: https://youtu.be/-NZO0LaA_Zs
(Extra) Live Musicbox Demo here: *https://youtu.be/Pc_jBn6hHbY*

Streaming audio-to-MIDI melody extraction. Runs in constant memory, single-threaded, with no machine learning (ML) involved.

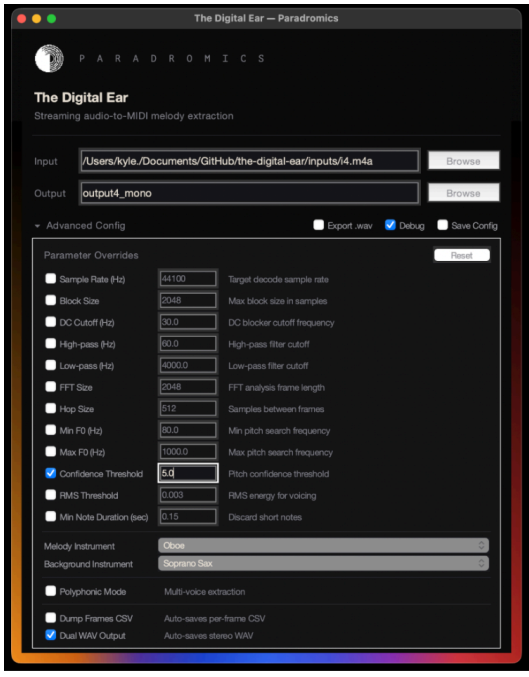
Requirements

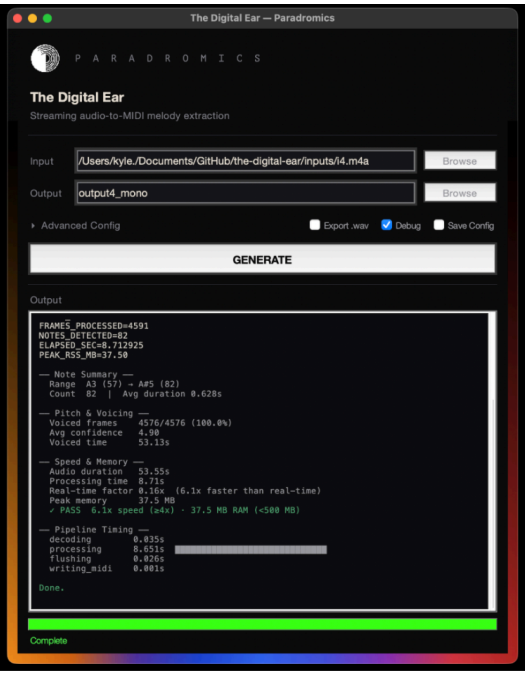
- Python 3.7+
- NumPy
- `ffmpeg` on your PATH

Install required packages:
> pip install numpy psutil

Run the GUI

1. Run the command:
   python gui.py
2. Select an input file, and click **Generate**.
3. The output MIDI file will appear in the `outputs/` directory.





Additional Export Options:
- MIDI -> .wav export
- Dual Stereo overlay export
- Polyphonic output
- CSV frame dump (for debugging)

gui.py icon in taskbar

Run from the Command Line

```
# Extract melody to MIDI
> python main.py --in "Input 1.m4a" --out output.mid

# Side-by-side WAV (original left, synth right) — for listening comparison
> python main.py --in "Input 1.m4a" --out output.mid --dual output_dual.wav

# Polyphonic mode (up to 3 voices)
> python main.py --in "Input 1.m4a" --out output.mid --poly

# All together
> python main.py --in "Input 1.m4a" --out output.mid --poly --dual output_dual.wav --debug
```

Pre-generated outputs for all four inputs are available in `outputs/Output1/` through `outputs/Output4/`.

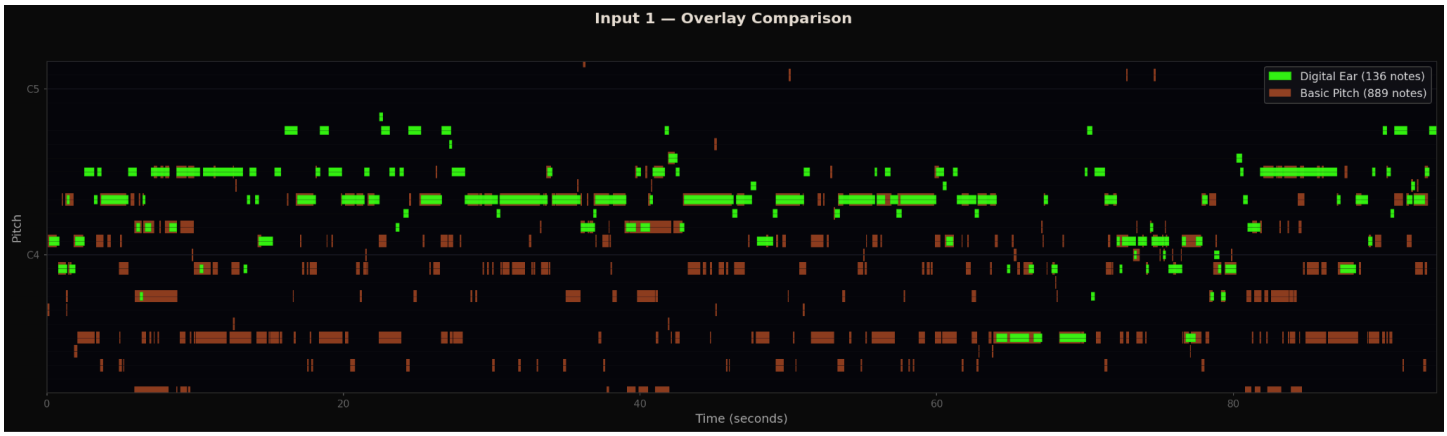
Results

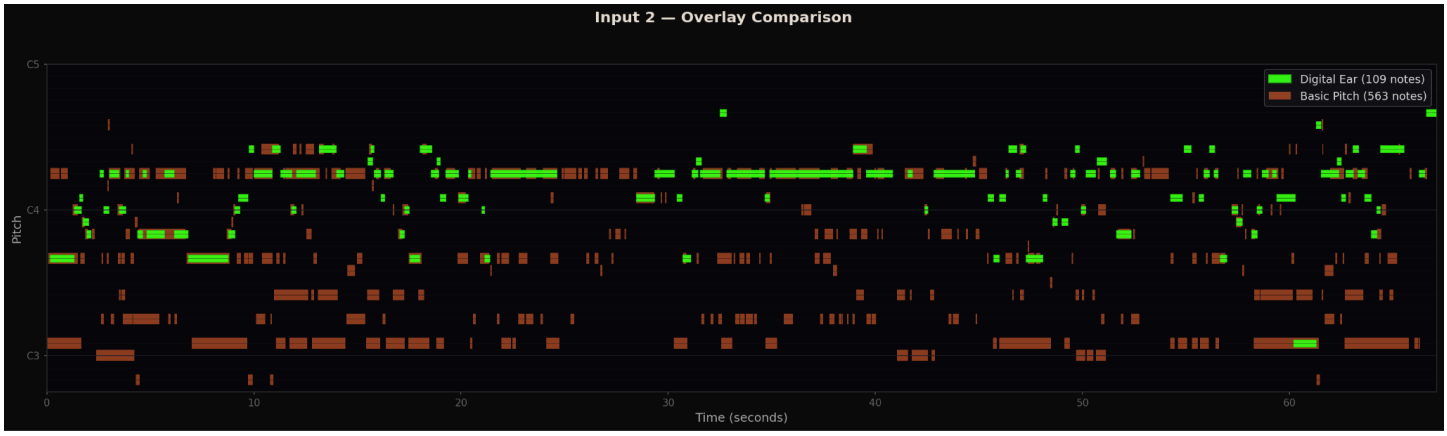
Digital Ear's melody output was compared against Spotify's Basic Pitch (a state-of-the-art polyphonic audio-to-MIDI model) using a note-level matching process with a ±0.3 second onset tolerance.

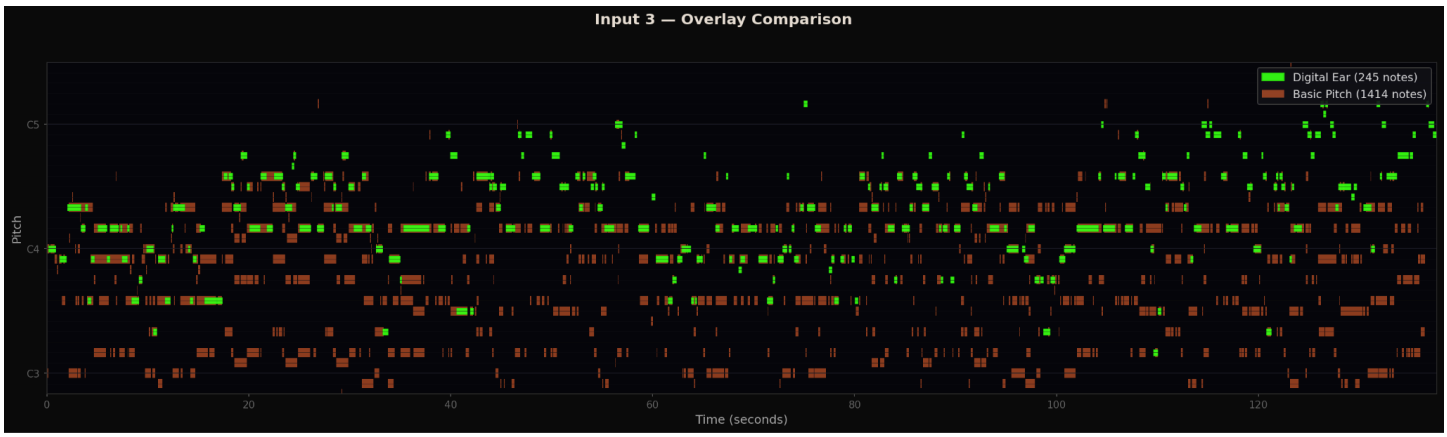
Input	Digital Ear	Basic Pitch	Strict Precision	Octave-Tolerant Precision
1	136 notes	889 notes	56.6%	92.6%
2	109 notes	563 notes	67.0%	97.2%
3	245 notes	1,414 notes	74.3%	98.8%
4	82 notes	936 notes	82.9%	93.9%
Average			**70.2%**	**95.6%**

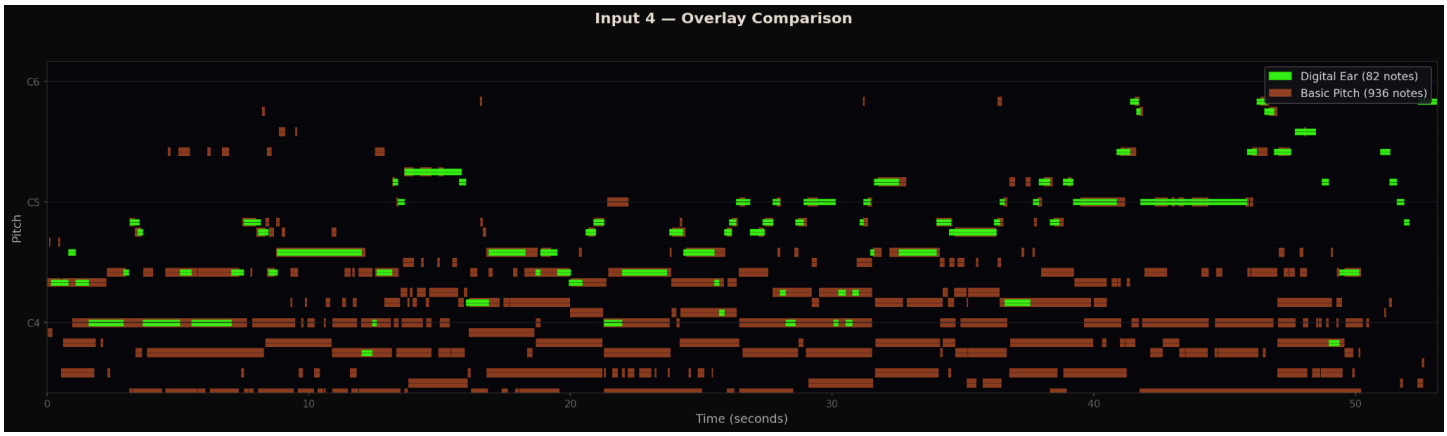
The average **95.6% octave-tolerant precision** confirms that nearly every note Digital Ear finds is a real note in the music.

- Recall is low by design: Digital Ear extracts the **melody only** (82–245 notes), whereas Basic Pitch captures every instrument (563–1,414 notes).
- Precision in this context: "Of the notes we found, how many were correct?"









Key:
Blue = Digital Ear (melody only) **Orange** = Basic Pitch (all instruments)

High Level Pipeline:

The process for melody extraction is as follows:
Audio in → Preprocess → HPSS → Frame → Pitch Detection → Viterbi → Note Tracking → MIDI out

Every stage is streaming and has bounded memory.

Metric	Value
End-to-end latency	~753 ms
Memory (constant)	~31 MB regardless of input length
Speed	4–7× faster than real-time
External dependencies	NumPy + `ffmpeg` only

See `README.md` for full technical details, academic references, and architecture breakdown.
Or visit the detailed Google Slides presentation here: The Digital Ear

(Extra) Live Musicbox Simulation (See here: *https://youtu.be/Pc_jBn6hHbY*)

Run:

```
> python live_musicbox.py
```